Systems programming lesson 7

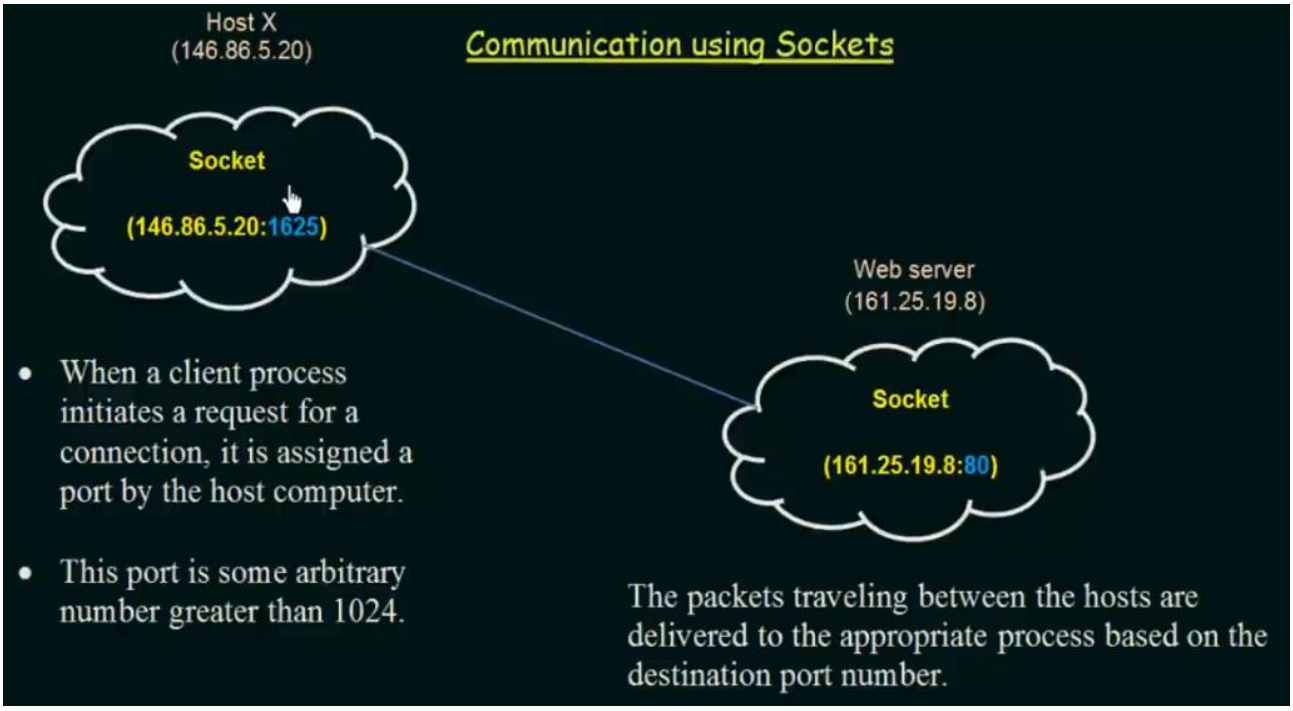
COM 312: BSc Computer science

Topics area coverage.

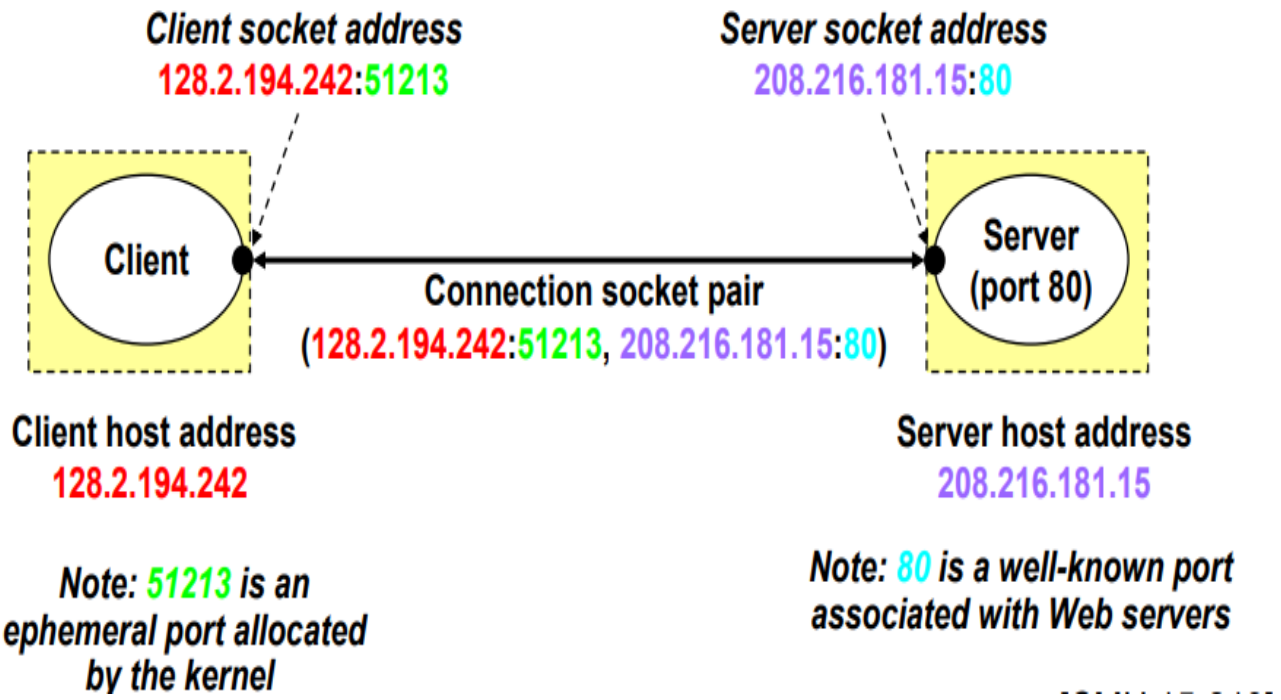
- **Socket programming in unix**
- **Socket system call APIs**

Socket programming

- **Socket is a virtual endpoint of any kind of network communication done between two hosts over in a network.** Sockets can be used in many languages like Java, C++, C etc.
- The socket API on Linux is similar to BSD/UNIX sockets from which it has evolved. Although over time the API has become slightly different at few places and now the newer official standard is POSIX sockets API which is same as BSD sockets.
    - A socket is identified by an IP address concatenated with a port number.
    - The server waits for incoming client requests by listening to a specified port. Once a request is received, the server accepts a connection from the client socket to complete the connection.
    - Servers implementing specific services (such as telnet, ftp, and http) listens to well-known ports (a telnet server listens to port 23, an ftp server listens port 21 and a web or http, server listens to port 80).
    - All ports below 1024 are considered well known; we can use them to implement standard services


Host X
(146.86.5.20)
Communication using Sockets
Socket
(146.86.5.20:1625)
Web server
(161.25.19.8)
Socket
(161.25.19.8:80)
When a client process initiates a request for a connection, it is assigned a port by the host computer.
This port is some arbitrary number greater than 1024.
The packets traveling between the hosts are delivered to the appropriate process based on the destination port number.

Example 2


Client socket address
128.2.194.242:51213
Server socket address
208.216.181.15:80
Client
Server
(port 80)
Connection socket pair
(128.2.194.242:51213, 208.216.181.15:80)
Client host address
128.2.194.242
Server host address
208.216.181.15
Note: 51213 is an ephemeral port allocated by the kernel
Note: 80 is a well-known port associated with Web servers

How to create a socket?

To create a socket, we can use the Socket function. Here is the sample code.

socket_tutorial.c

```c
#include<stdio.h>
#include<sys/socket.h>

int main(int argc , char *argv[])
{
    int socket_desc;
    socket_desc = socket(AF_INET , SOCK_STREAM , 0);

    if (socket_desc == -1)
    {
        printf("Could not create socket");
    }

    return 0;
}
```

Code explanation

socket () creates a socket and returns a descriptor which can be used in other functions.

```c
int main(int argc , char *argv[])
```

- **argc** (ARGument Count) is int and stores number of command-line arguments passed by the user including the name of the program.
- The value of argc should be non-negative.
- **Arg v** (ARGument Vector) is array of character pointers listing all the arguments.

The above code will create a socket with following properties.

- **Address Family: AF_INET (**IP Version 4**)**
- **Type: SOCK_STREAM (**TCP Protocol**)**
- **Protocol: 0 (**Default IP Protocol**)**
- Instead of using SOCK_STREAM type of sockets there is another type called SOCK_DGRAM which indicates the UDP protocol.

Types of Sockets:

There are two types of Sockets: the datagram socket and the stream socket.

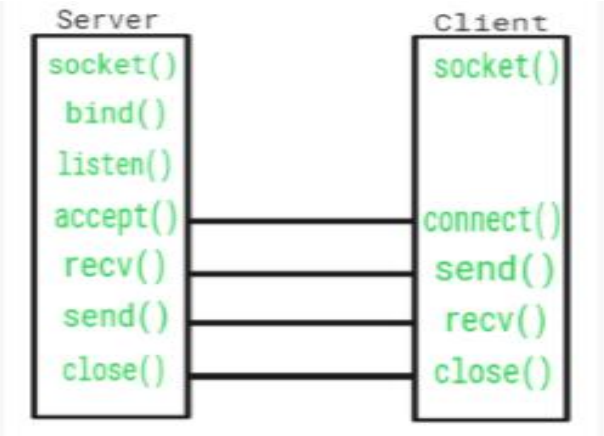
Datagram Socket:

This is a type of network which has connection less point for sending and receiving packets. It is similar to mailbox. The letters (data) posted into the box are collected and delivered (transmitted) to a letterbox (receiving socket).

Stream Socket

In Computer operating system, a stream socket is type of interposes communications socket or network socket which provides a connection-oriented, sequenced, and unique flow of data without record boundaries with well-defined mechanisms for creating and destroying connections and for detecting errors. It is similar to phone. A connection is established between the phones (two ends) and a conversation (transfer of data) takes place.

example 1 **how** network socket uses system calls API to communicate



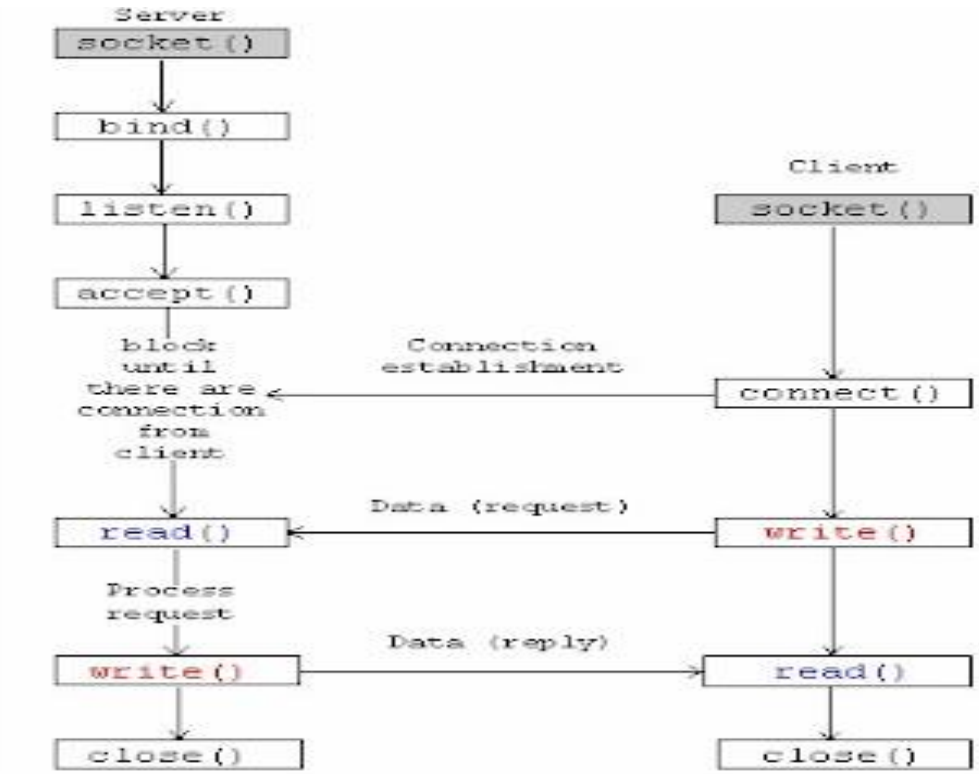
Function Call	Description
Create ()	To create a socket
Bind ()	It's a socket identification like a telephone number to contact
Listen ()	Ready to receive a connection
Connect ()	Ready to act as a sender

Accept ()	Confirmation, it is like accepting to receive a call from a sender
Write ()	To send data
Read ()	To receive data
Close ()	To close a connection

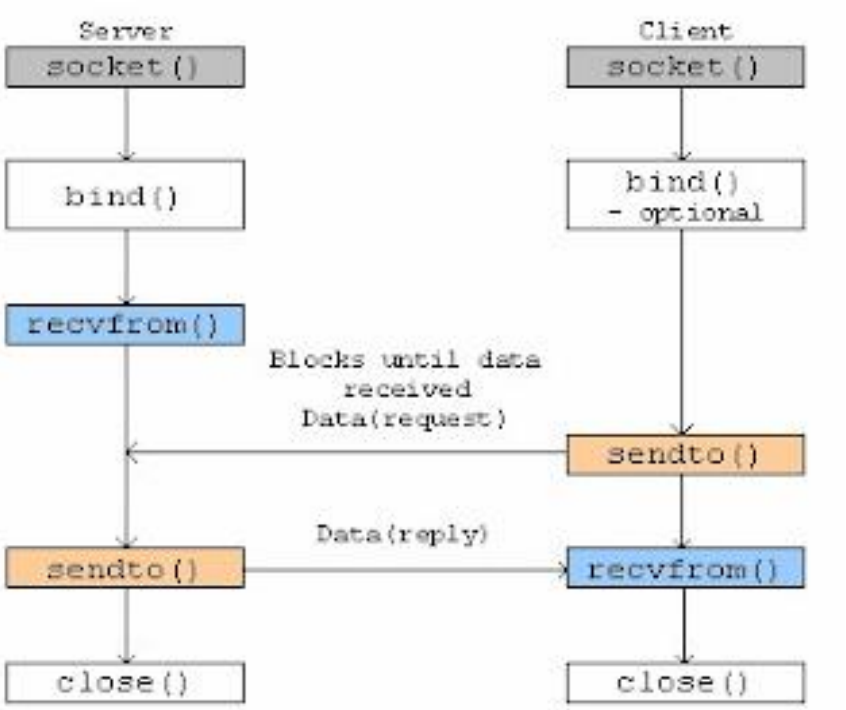
how client and server processes across a network communicate using **socket API**

1. **client/server relationship of the socket APIs for connection-oriented protocol (TCP)**
2. **client/server relationship of the socket APIs for a connectionless protocol (UDP)**

client/server relationship of the socket APIs for connection-oriented protocol (TCP)



client/server relationship of the socket APIs for a connectionless protocol (UDP)



Port vs. Socket

We now know the basics of port and socket. Let's see the differences between a port and a socket:

Port	Socket
Port specifies a number that is used by a program in a computer.	A socket is a combination of IP address and port number.
A program running on different computers can use the same port number. Hence port numbers can't be used to identify a computer uniquely.	It identifies a computer as well as a program within the computer uniquely.
Port number is used in the transport layer.	Sockets are involved in the application layer. A socket is an interface between the transport and application layer.
Port uses a socket to drop the data to a correct application.	A server and a client uses a socket to keep an eye on the data request and responses.

How Do Sockets Work?

Sockets simply provide the actual hardware for moving data around. TCP-based sockets are called stream sockets, where all data will arrive in order. UDP-based sockets are datagram sockets, where order (or even delivery) isn't guaranteed. There are also raw sockets, which don't have any restrictions, and are used for implementing different protocols and utilities that need to inspect low-level network traffic, like **Wireshark.**

Sockets usually still use TCP or UDP, **as they aren't anything special other than a fancy pipe within the kernel.** TCP and UDP are transport protocols that define how data gets from place to place but don't really care about what the data is. TCP and UDP provide the platform for most other protocols like FTP, SMTP, and RDP, which operate at higher levels.

It is possible for an application to use a slightly different implementation of TCP; stream sockets use the SOCK_STREAM protocol, which is what TCP also uses for transport almost all the time, and while they're basically interchangeable, they're technically slightly different. Though this is low-level stuff and isn't really something you'll have to worry about, just know that most traffic sent through UNIX domain sockets is TCP- or UDP-based, or at least quite similar to it, and TCP sent over UNIX domain sockets is faster than TCP over network interfaces like ports.

Socket Use in Practice

The biggest example of this is Redis, an extremely fast key-value store that operates entirely within memory. Redis is frequently used on the same server that's accessing it, so you'll usually be able to use sockets. At such low levels and with how fast Redis is, sockets provide a 25%-performance boost in some synthetic benchmarks.

If you're connecting to a MySQL database, you can also use a socket. Usually you'd connect to host: port from a remote system, but if you're connecting to a database on

the same server (for example, an REST API accessing a database), you can use sockets for a speedup. This won't affect normal use, but is very noticeable when under load, over 20% on a high end 24 core with 128 concurrent users and a million queries per second. Whether or not you'll see a benefit from sockets is a different story, but at that point you'll likely want to be looking into replication and load balancing anyway.